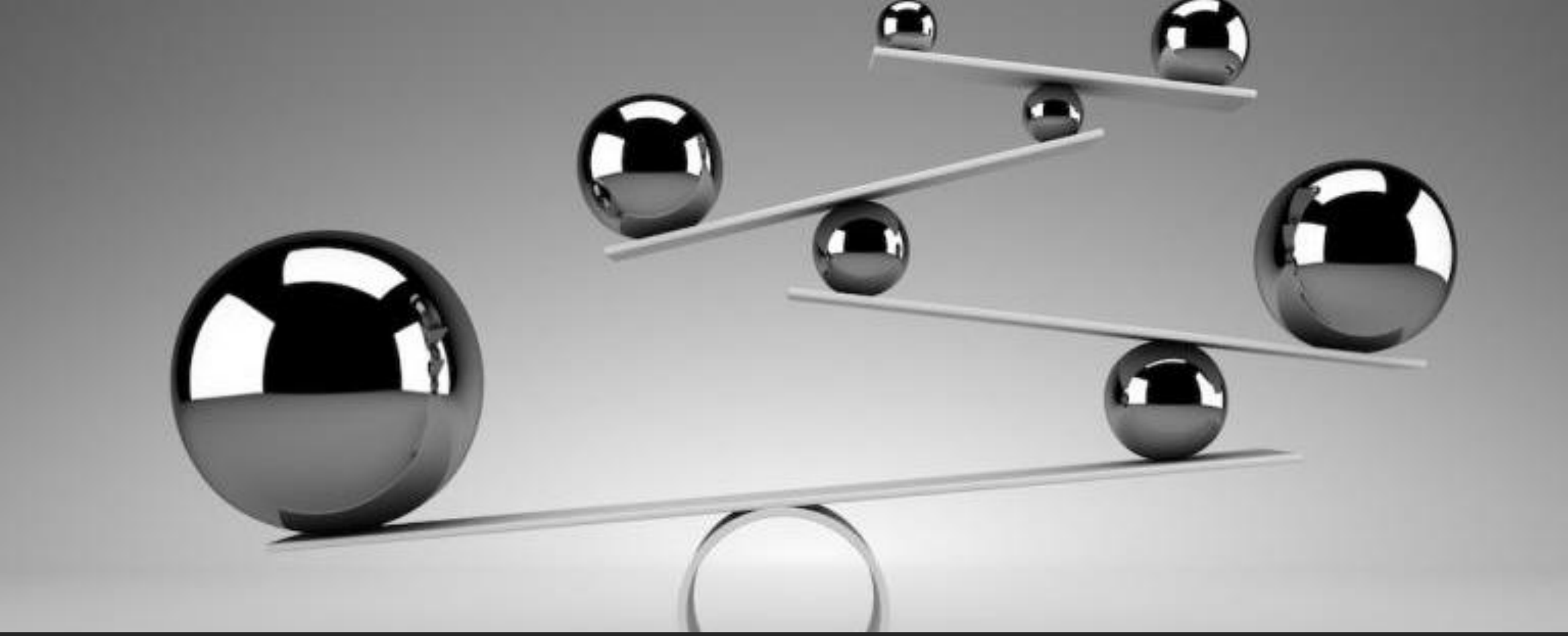
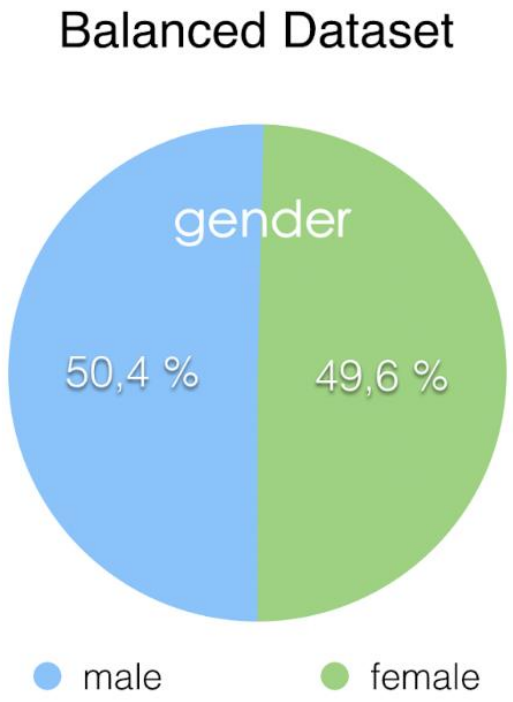
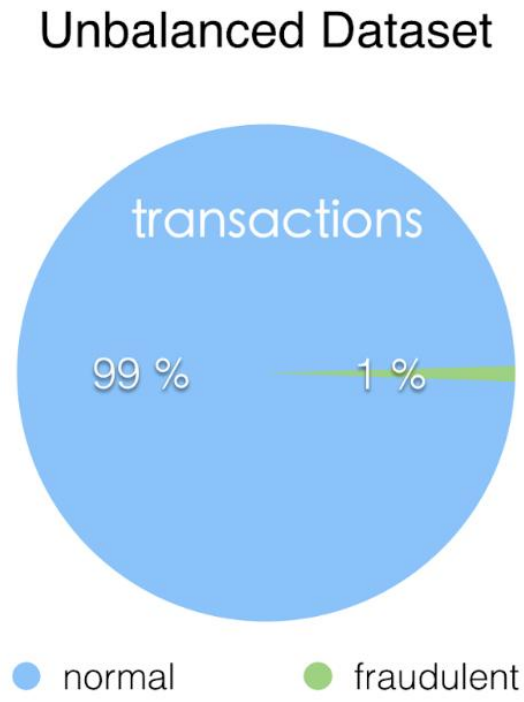


Classification of Imbalanced Data

CHEN HAJAJ

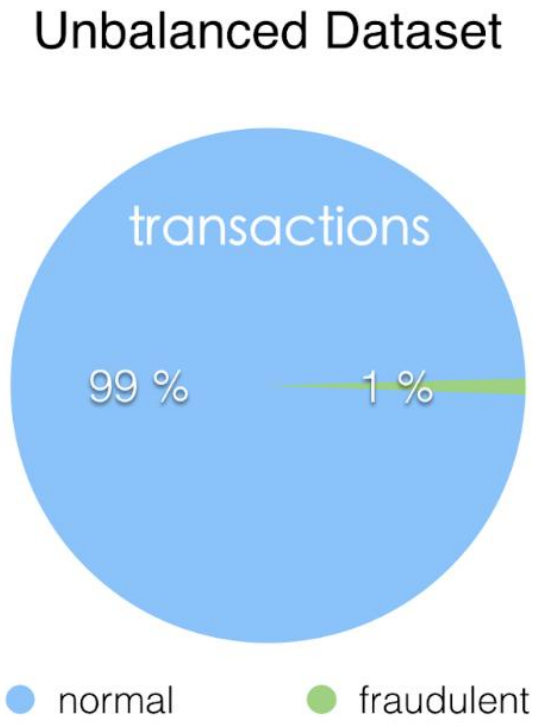
CREDITS TO SATYAM KUMAR





Classes are not always balanced

Fixed prediction



Predict that every transaction is normal

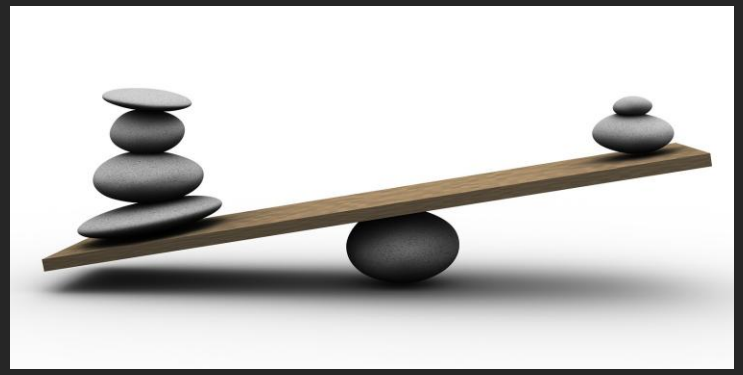
Our model achieves an accuracy of 0.99

What about recall?

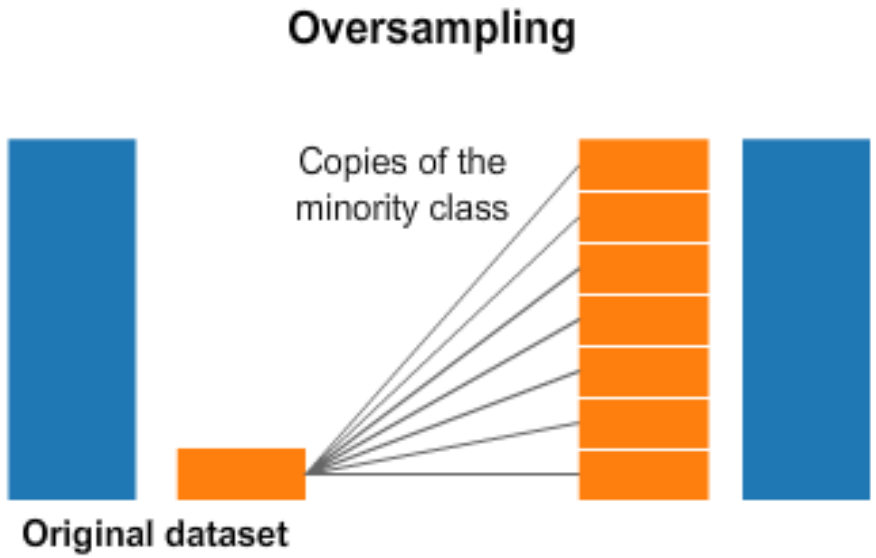
Agenda

1. Upsampling Minority Class
2. Downsampling Majority Class
3. Generate Synthetic Data
4. Combine Upsampling & Downsampling Techniques
5. Balanced Class Weight

Imbalanced data



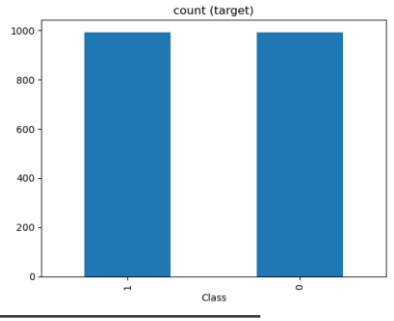

```python
import pandas as pd
import numpy as np

# make a dataset
np.random.seed(0)
n = 1000
X = np.random.randn(n, 2)

# make a target
y = np.random.choice([0, 1], n, p=[0.99, 0.01])

# make a DataFrame
data = pd.DataFrame(X, columns=['X1', 'X2'])
data['Class'] = y

# class count
class_count_0, class_count_1 = data['Class'].value_counts()

# Separate class
class_0 = data[data['Class'] == 0]
class_1 = data[data['Class'] == 1]

# print the shape of the class
print('class 0:', class_0.shape)
print('class 1:', class_1.shape)
```



Upsampling Minority Class

```python
class_1_over = class_1.sample(class_count_0, replace=True)
test_over = pd.concat([class_1_over, class_0], axis=0)

print("total class of 1 and 0:", test_over['Class'].value_counts())
# plot the count after under-sampeling
test_over['Class'].value_counts().plot(kind='bar', title='count (target)')
```

Random Over-Sampling

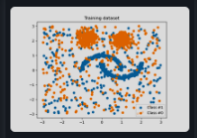


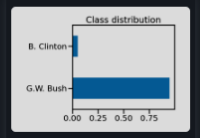
Examples based on real world datasets

Examples which use real-word dataset.


Multiclass classification with under-sampling


Example of topic classification in text documents


Customized sampler to implement an outlier rejections estimator



Benchmark over-sampling methods in a face recognition task


Porto Seguro: balancing samples in mini-batches with Keras


Fitting model on imbalanced datasets and how to fight bias

https://imbalanced-learn.org/stable/

imbalanced-learn documentation

Date: Dec 20, 2024 **Version**: 0.13.0

Useful links: Binary Installers | Source Repository | Issues & Ideas | Q&A Support

Imbalanced-learn (imported as `imblearn`) is an open source, MIT-licensed library relying on scikit-learn (imported as `sklearn`) and provides tools when dealing with classification with imbalanced classes.

Getting started

Check out the getting started guides to install `imbalanced-learn`. Some extra information to get started with a new contribution is also provided.

To the installation guideline

User guide

The user guide provides in-depth information on the key concepts of `imbalanced-learn` with useful background information and explanation.

To the user guide

API reference

The reference guide contains a detailed description of the `imbalanced-learn` API. To known more about methods parameters.

To the reference guide

Examples

The gallery of examples is a good place to see `imbalanced-learn` in action. Select an example and dive in.

To the gallery of examples

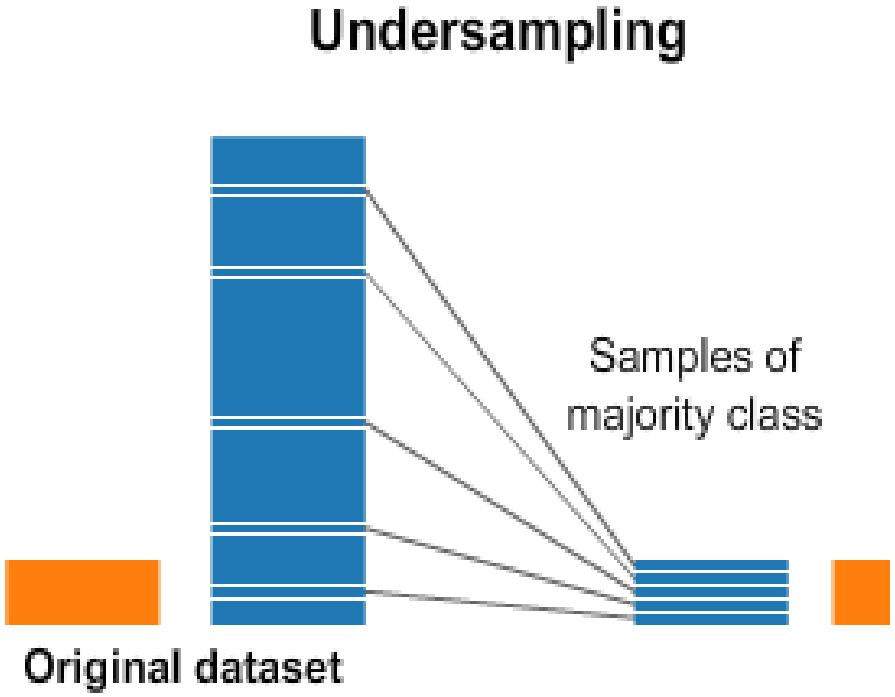
```python
from imblearn.over_sampling import RandomOverSampler
import collections
ros = RandomOverSampler(random_state=42)
# fit predictor and target variable
x_ros, y_ros = ros.fit_resample(X, y)
print('Original dataset shape', collections.Counter(y))
print('Resample dataset shape', collections.Counter(y_ros))
# Original dataset shape Counter({0: 992, 1: 8})
# Resample dataset shape Counter({0: 992, 1: 992})
```

Random over-sampling with imblearn

Any other simple options?

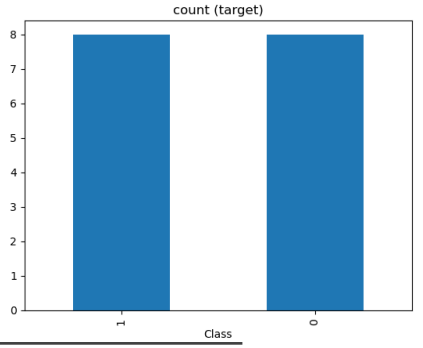
Downsampling Majority Class

- Random Undersampling
- Tomek Links
- NearMiss Sampling
- Edited Nearest Neighbors (ENN)



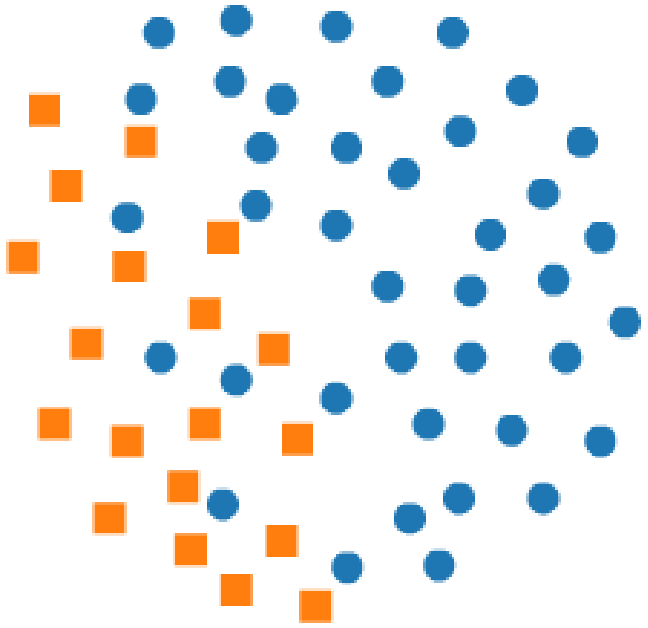
```python
class_0_under = class_0.sample(class_count_1)
test_under = pd.concat([class_1, class_0_under], axis=0)

print("total class of 1 and 0:", test_under['Class'].value_counts())
# plot the count after under-sampeling
test_under['Class'].value_counts().plot(kind='bar', title='count (target)')
```

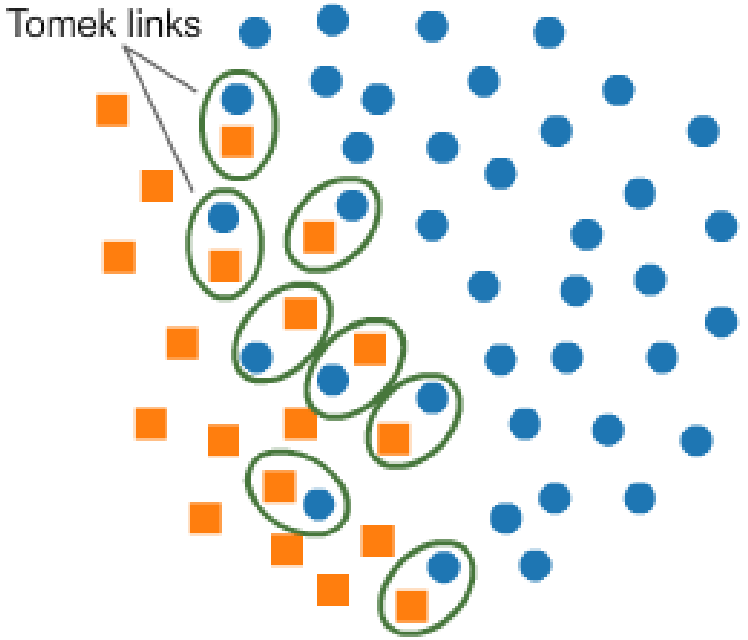
Random Under-Sampling

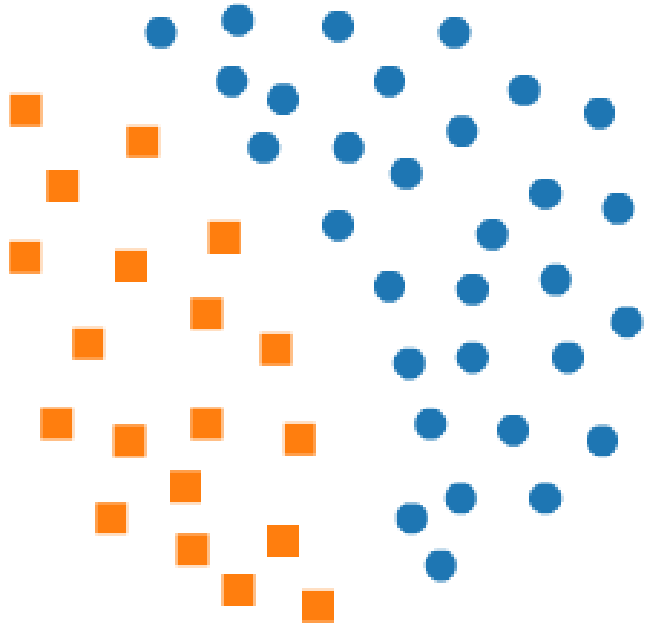


```python
# import library
from collections import Counter
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(random_state=42, replacement=True)
# fit predictor and target variable
X_rus, y_rus = rus.fit_resample(X, y)

print('Original dataset shape:', Counter(y))
print('Resample dataset shape', Counter(y_rus))
# Original dataset shape Counter({0: 992, 1: 8})
# Resample dataset shape Counter({0: 8, 1: 8})
```

Random under-sampling with imblearn







Tomek links

Two points form a Tomek link if:

1. They belong to **different classes**
2. **Each** of the two points is the **nearest neighbor** of the other point

The rationale here is that such points don't help make the decision boundary better (e.g., may make overfitting easier) and that they may be noise.

```python
# import library
from imblearn.under_sampling import TomekLinks
tl = TomekLinks(sampling_strategy='majority')

# fit predictor and target variable
x_tl, y_tl = tl.fit_resample(X, y)

print('Original dataset shape', Counter(y))
print('Resample dataset shape', Counter(y_tl))
# Original dataset shape Counter({0: 992, 1: 8})
# Resample dataset shape Counter({0: 985, 1: 8})
```

Tomek links

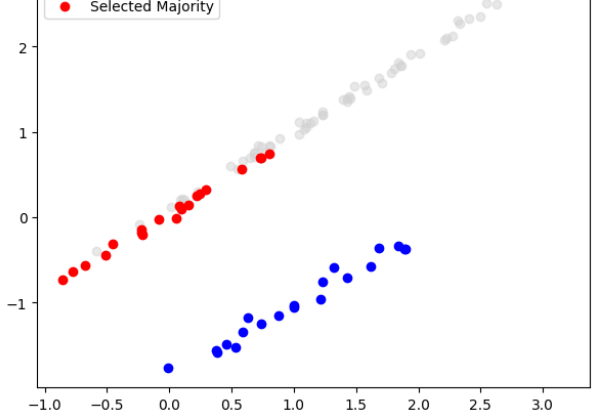
NearMiss

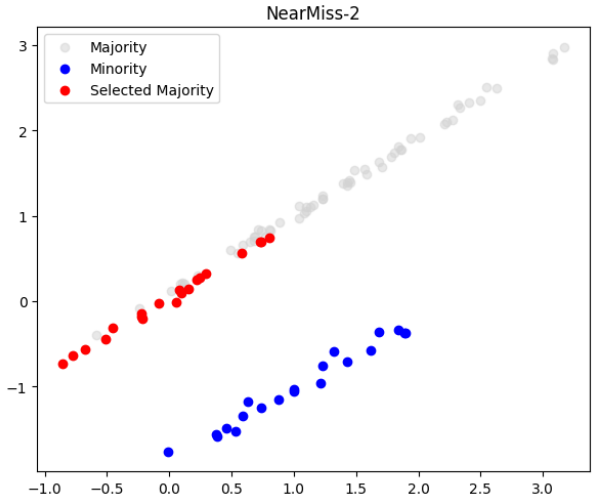
- **NearMiss-1**: Selects majority class samples that are closest to the minority class samples.
- **NearMiss-2**: Chooses majority samples with the smallest average distance to multiple nearest minority samples.
- **NearMiss-3**: For each minority sample, selects a fixed number of nearest majority samples

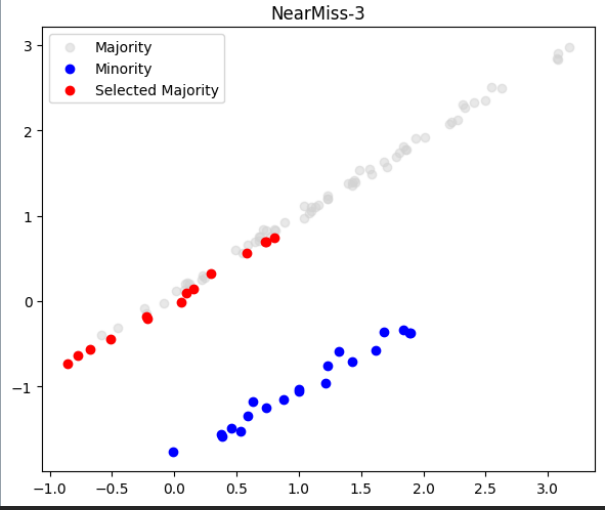
```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_classification
from sklearn.neighbors import NearestNeighbors

# Generate synthetic 2D data
X, y = make_classification(n_samples=100, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, weights=[0.2, 0.8],
                           class_sep=1.0, random_state=42)

# Separate the classes
minority = X[y == 0]
majority = X[y == 1]
```

```python
# Plot
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
titles = ["NearMiss-1", "NearMiss-2", "NearMiss-3"]
selected_indices = [nm1_indices, nm2_indices, nm3_indices]

for i, ax in enumerate(axes):
    ax.scatter(majority[:, 0], majority[:, 1], color='lightgray', label='Majority', alpha=0.5)
    ax.scatter(minority[:, 0], minority[:, 1], color='blue', label='Minority')
    ax.scatter(majority[selected_indices[i], 0], majority[selected_indices[i], 1],
               color='red', label='Selected Majority')
    ax.set_title(titles[i])
    ax.legend()

plt.tight_layout()
plt.show()
```

```python

# Parameters
k = 3

# NearMiss-1: Select majority samples closest to any minority sample
nn = NearestNeighbors(n_neighbors=k)
nn.fit(minority)
distances, _ = nn.kneighbors(majority)
avg_distance_nm1 = distances.mean(axis=1)
nm1_indices = np.argsort(avg_distance_nm1)[:len(minority)]

# NearMiss-2: Select majority samples with smallest average distance to k nearest minority samples
nn = NearestNeighbors(n_neighbors=k)
nn.fit(minority)
distances, _ = nn.kneighbors(majority)
avg_distance_nm2 = distances.mean(axis=1)
nm2_indices = np.argsort(avg_distance_nm2)[:len(minority)]

# NearMiss-3: For each minority sample, find k nearest majority samples
nn = NearestNeighbors(n_neighbors=k)
nn.fit(majority)
_, indices_nm3 = nn.kneighbors(minority)
nm3_indices = np.unique(indices_nm3.flatten())

```







```
1  from imblearn.under_sampling import NearMiss
2  nm = NearMiss(version=1/2/3, n_neighbors=5)
3  x_nm, y_nm = nm.fit_resample(x, y)
4  print('Original dataset shape:', Counter(y))
5  print('Resample dataset shape:', Counter(y_nm))
```

NearMiss - Code

ENN (Edited Nearest Neighbors)

Edited Nearest Neighbours (ENN) cleans the dataset by removing samples from the majority class that don't fit well with their neighbors.

For each majority sample, it looks at its nearest neighbors:

If the sample **doesn't match** its neighbors based on a chosen rule, it is **removed**.

There are two selection rules:

1. **'mode'** (default): keep the sample only if **most neighbors** belong to the same class.
2. **'all'**: keep the sample only if **all neighbors** belong to the same class.

```python
from collections import Counter
from sklearn.datasets import make_classification
from imblearn.under_sampling import EditedNearestNeighbours
X, y = make_classification(n_classes=2, class_sep=2,
weights=[0.1, 0.9], n_informative=3, n_redundant=1, flip_y=0,
n_features=20, n_clusters_per_class=1, n_samples=1000, random_state=10)
print('Original dataset shape %s' % Counter(y))
#Original dataset shape Counter({1: 900, 0: 100})
enn = EditedNearestNeighbours(kind_sel='all', n_neighbors=5)
X_res, y_res = enn.fit_resample(X, y)
print('Resampled dataset shape %s' % Counter(y_res))
#Resampled dataset shape Counter({1: 885, 0: 100})
enn = EditedNearestNeighbours(kind_sel='mode', n_neighbors=5)
X_res, y_res = enn.fit_resample(X, y)
print('Resampled dataset shape %s' % Counter(y_res))
#Resampled dataset shape Counter({1: 898, 0: 100})
```

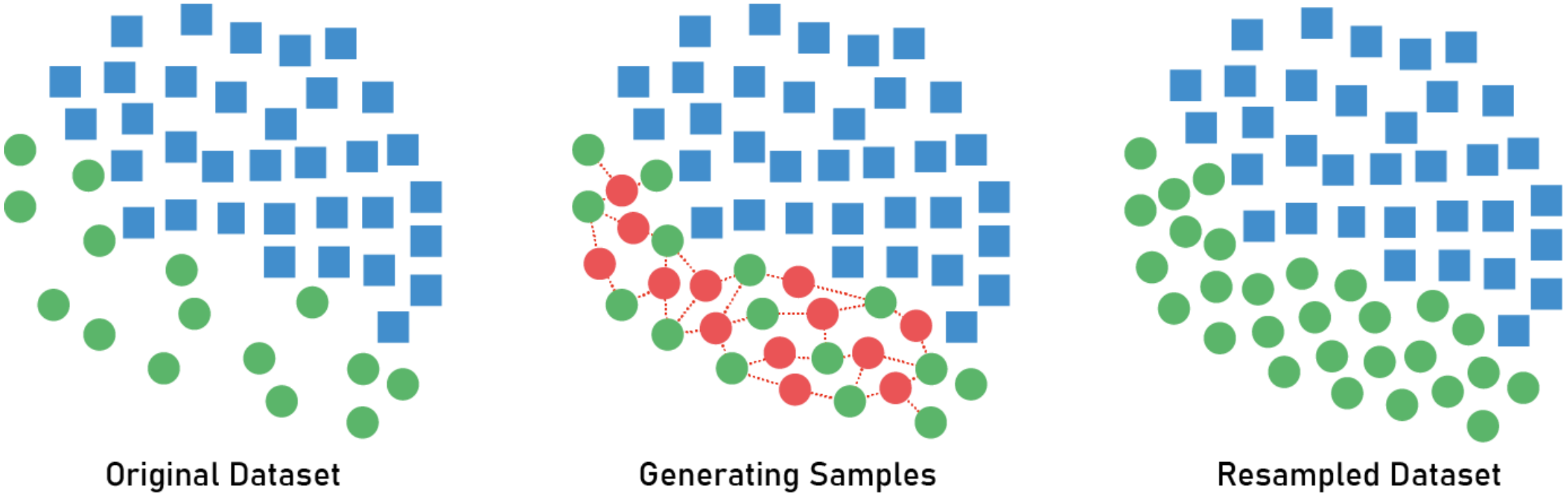
Code example

Synthetic Minority Oversampling Technique (SMOTE)

Generating synthetic data points of minority samples is a type of oversampling technique

The idea is to generate synthetic data points of minority class samples in the nearby region or neighborhood of minority class samples

Synthetic Minority Oversampling Technique



Original Dataset

Generating Samples

Resampled Dataset

Synthetic Minority Over-sampling Technique - SMOTE

SMOTE works by utilizing a k-nearest neighbor algorithm to create synthetic data

1. Identify the feature vector and its nearest neighbor
2. Compute the distance between the two sample points
3. Multiply the distance by a random number between 0 and 1
4. Identify a new point on the line segment at the computed distance
5. Repeat the process for identified feature vectors

```python
# import library
from imblearn.over_sampling import SMOTE
smote = SMOTE()
# fit predictor and target variable
x_smote, y_smote = smote.fit_resample(X, y)
print('Original dataset shape', Counter(y))
print('Resample dataset shape', Counter(y_smote))
# Original dataset shape Counter({1: 900, 0: 100})
# Resample dataset shape Counter({0: 900, 1: 900})
```

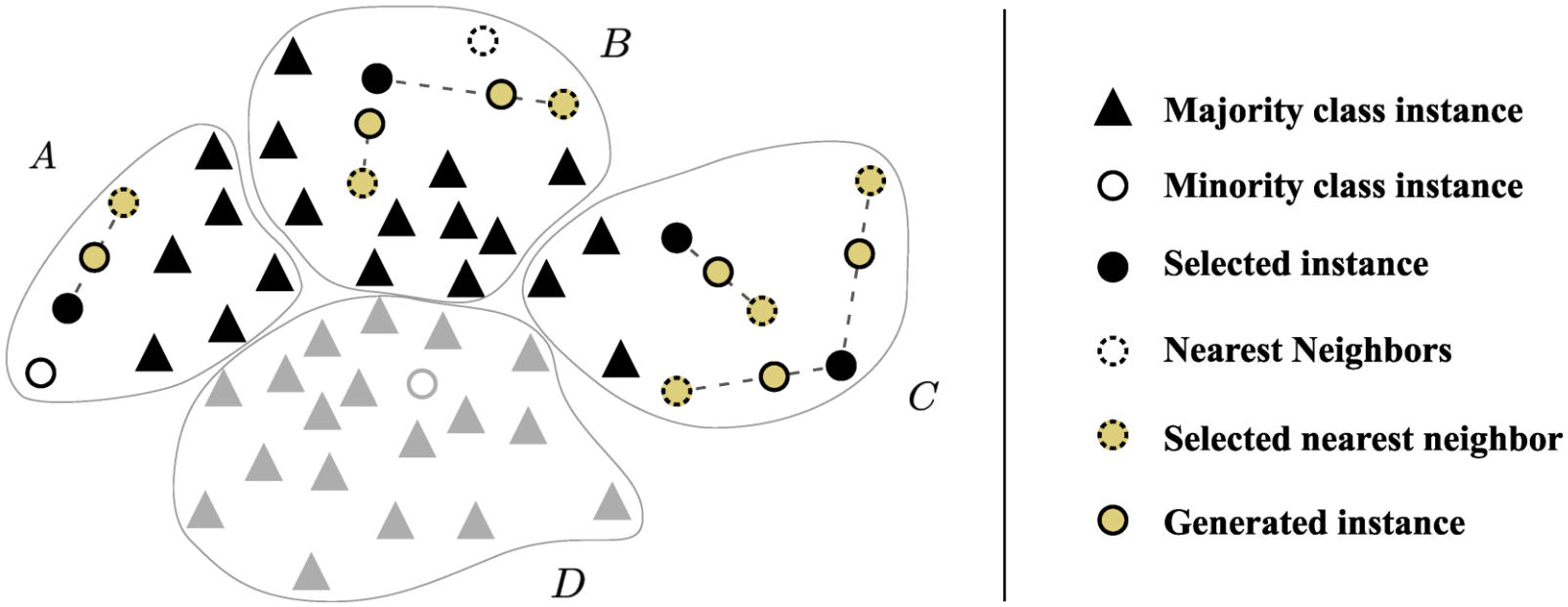
SMOTE - Code

K-means SMOTE

Aids classification by generating minority class samples in safe and crucial areas of the input space

K-Means SMOTE works in three steps:

1. Cluster the entire data using the k-means clustering algorithm
2. Select clusters that have a high number of minority class samples
3. Assign more synthetic samples to clusters where minority class samples are sparsely distributed



Borderline SMOTE

Due to the presence of some minority points or outliers within the region of majority class points, bridges of minority class points are created

In the Borderline SMOTE technique, only the minority examples near the borderline are over-sampled

classifies the minority class points into noise points, and border points

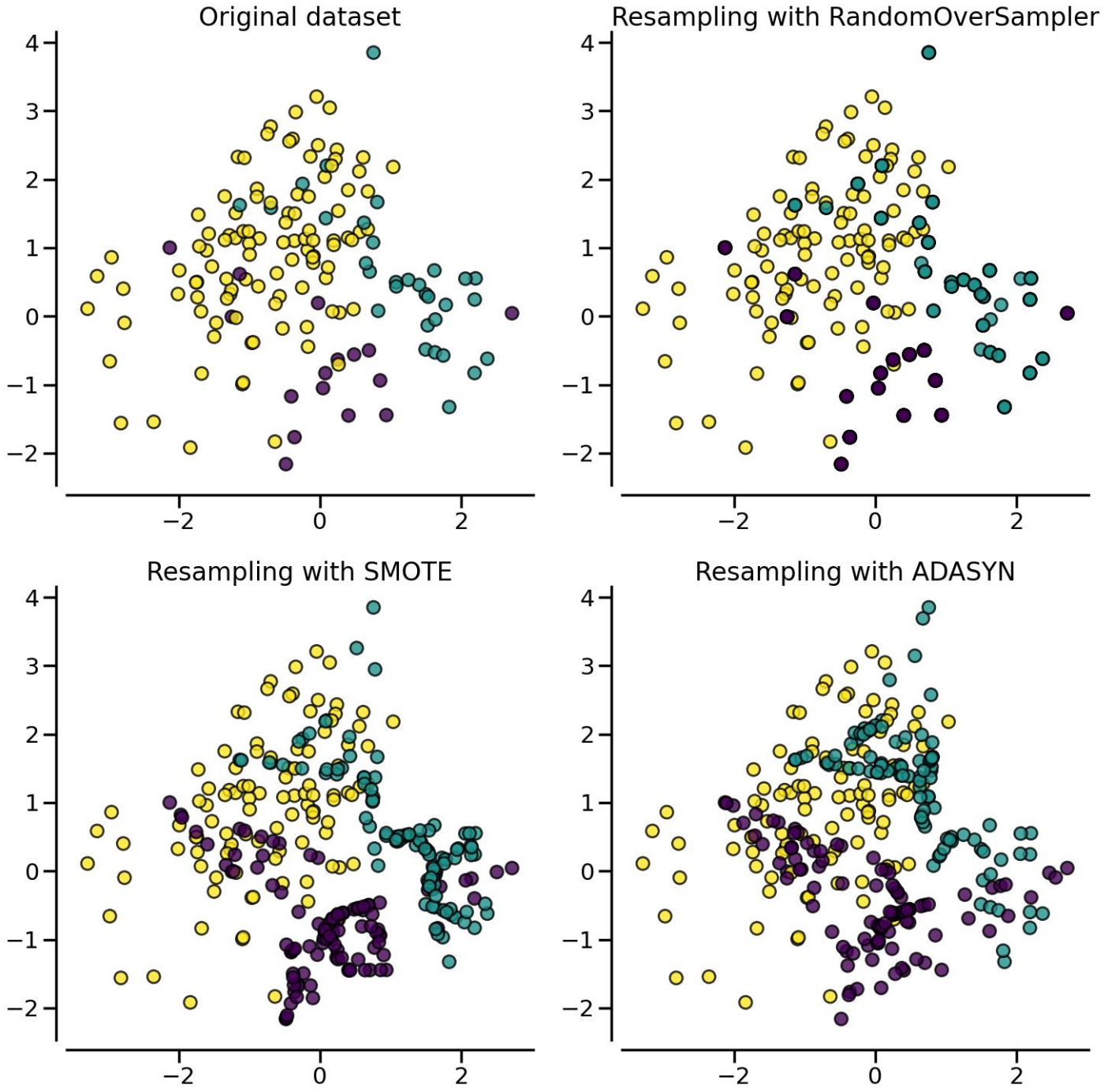
Noise points are minority class points that have most of the points as majority points in its neighborhood	Border points have both majority and minority class points in their neighborhood

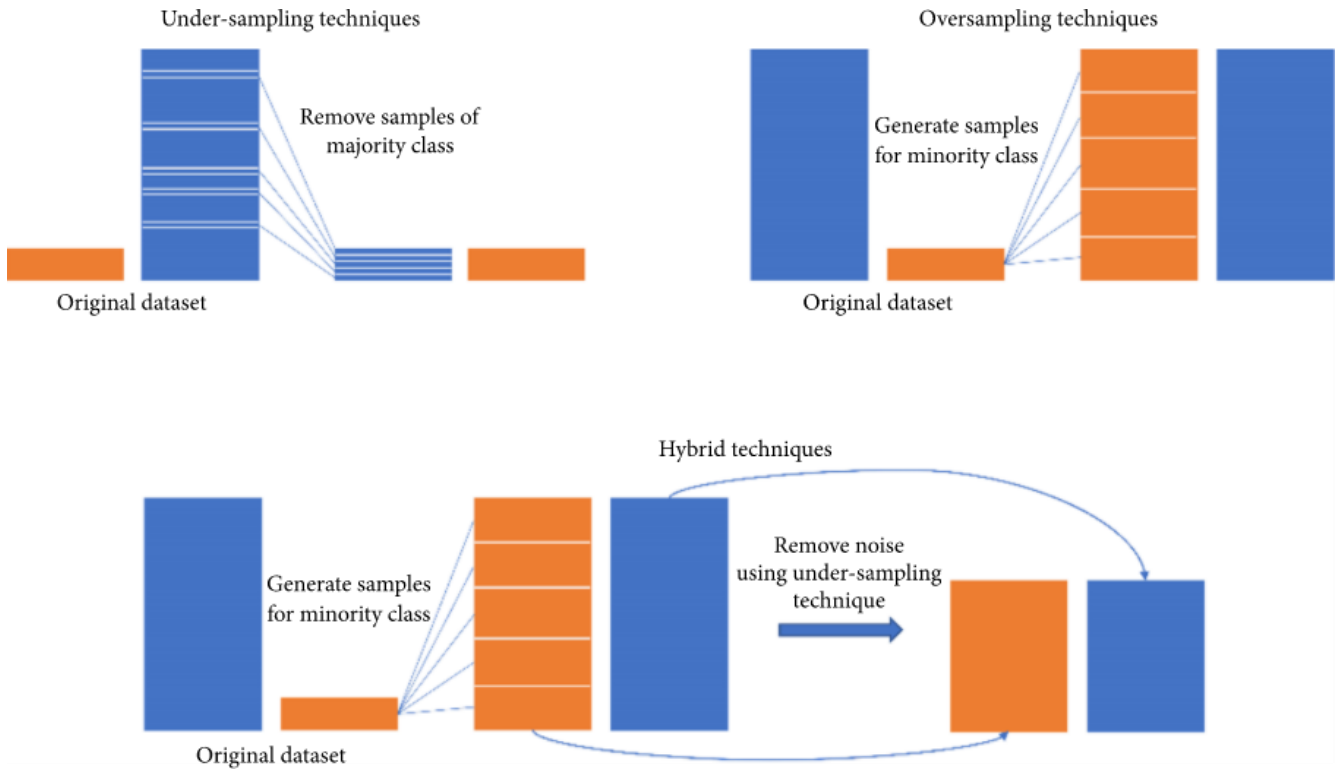
Adaptive Synthetic Sampling (ADASYN)

Borderline SMOTE gives more importance or creates synthetic points using only the extreme observations that are the border points and ignores the rest of minority class points

This problem is solved by the ADASYN algorithm, as it **creates synthetic data according to the data density**

SMOTE & ADASYN





Combine Oversampling and Undersampling Techniques

Hybrid

Undersampling techniques are not recommended as it removes the majority class data points.

Oversampling techniques are often considered better than undersampling techniques

Imblearn comes with the implementation of combined oversampling and undersampling techniques, such as:

- SMOTE-Tomek: SMOTE (oversampler) combined with TomekLinks (undersampler)
- SMOTE-ENN: SMOTE (oversampler) combined with ENN (undersampler)

```python
from collections import Counter
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=5000, n_features=2, n_informative=2, n_redundant=0, n_repeated=0,
n_classes=3, n_clusters_per_class=1, weights=[0.01, 0.05, 0.94], class_sep=0.8, random_state=0)
print(sorted(Counter(y).items()))
#[(0, 64), (1, 262), (2, 4674)]
from imblearn.combine import SMOTEENN
smote_enn = SMOTEENN(random_state=0)
X_resampled, y_resampled = smote_enn.fit_resample(X, y)
print(sorted(Counter(y_resampled).items()))
#[(0, 4060), (1, 4381), (2, 3502)]
from imblearn.combine import SMOTETomek
smote_tomek = SMOTETomek(random_state=0)
X_resampled, y_resampled = smote_tomek.fit_resample(X, y)
print(sorted(Counter(y_resampled).items()))
#[(0, 4499), (1, 4566), (2, 4413)]
```

Code example

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

# Decision Tree
dt = DecisionTreeClassifier(random_state=0)
print("Regular\n",classification_report(y_test, dt.fit(X_train, y_train).predict(X_test)))

dt = DecisionTreeClassifier(random_state=0, class_weight='balanced')
print("Balanced\n",classification_report(y_test, dt.fit(X_train, y_train).predict(X_test)))

class_weight = {0: 1, 1: 10, 2: 100}
dt = DecisionTreeClassifier(random_state=0, class_weight=class_weight)
print("Class weigth (1,10,100)\n",classification_report(y_test, dt.fit(X_train, y_train).predict(X_test)))
```

Using Class Weights

```
Regular
              precision    recall  f1-score   support

           0       0.62      0.68      0.65        19
           1       0.88      0.90      0.89        79
           2       0.99      0.99      0.99      1402

    accuracy                           0.98      1500
   macro avg       0.83      0.86      0.84      1500
weighted avg       0.98      0.98      0.98      1500

Balanced
              precision    recall  f1-score   support

           0       0.88      0.74      0.80        19
           1       0.89      0.82      0.86        79
           2       0.99      0.99      0.99      1402

    accuracy                           0.98      1500
   macro avg       0.92      0.85      0.88      1500
weighted avg       0.98      0.98      0.98      1500

Class weigth (1,10,100)
              precision    recall  f1-score   support

           0       0.52      0.68      0.59        19
           1       0.85      0.84      0.84        79
           2       0.99      0.98      0.99      1402

    accuracy                           0.97      1500
   macro avg       0.78      0.83      0.81      1500
weighted avg       0.97      0.97      0.97      1500
```

Results